Introduction to Agile Development

(Short Notes)

1. What is Agile ?

- Agile is a modern software development approach.
- Focuses on flexibility, customer collaboration and rapid delivery of working software.
- Develops the product in small increments (iterations).
- Can quickly respond to changes in requirements.
- Values people, communication and working software over strict documentation.

2. Agile Manifesto

- Created in 2001 by a group of 17 software developers.
- Provides 4 key values and 12 principles.
- Focuses on a more flexible and human-centric approach to development.

4 Key Values		
Individuals and interactions	over	Processes and tools
Working software	over	Comprehensive documentation
Customer collaboration	over	Contract negotiation
Responding to change	over	Following a plan

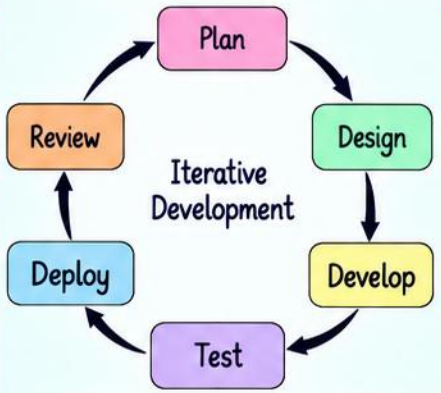
3. Agile Principles (Summary)

- There are 12 Agile principles. Some important ones are:
- Satisfy the customer through early and continuous delivery.
- Welcome changing requirements.
- Deliver working software frequently.
- Business people and developers work together.
- Build projects around motivated individuals.
- Keep it simple.
- Regularly reflect and adapt.
- Focus on continuous improvement.

4. Agile vs Traditional (Waterfall)

Aspect	Traditional (Waterfall)	Agile
Approach	Plan-driven	Iterative and adaptive
Requirements	Fixed early	Can change anytime
Development	Sequential	Incremental (iterations)
Customer involvement	Limited	Continuous
Documentation	Heavy	Lightweight
Delivery	At the end	Frequent working software
Flexibility	Low	High

5. Agile Process (General View)



Repeat the cycle to deliver improved versions of the product.

6. Benefits of Agile

- Faster delivery of working software.
- Can easily adapt to changes.
- Higher customer satisfaction.
- Better communication and collaboration.
- Early identification and resolution of risks.
- Improved product quality.
- Continuous improvement.

7. Challenges of Agile

- Requires active customer involvement.
- May be difficult with large or distributed teams.
- Less emphasis on documentation.
- Requires experienced and committed team members.
- Not always suitable for highly regulated projects.

8. Key Takeaways

- Agile is a flexible, iterative approach.
- Based on Agile Manifesto values and principles.
- Delivers working software in small increments.
- Focuses on people, collaboration and customer satisfaction.
- Not a one-size-fits-all solution.
- Choose Agile when requirements may change and quick delivery is important.

Scrum Framework (Basics)

(Short Notes)

1. What is Scrum ?

- Scrum is a popular Agile framework for managing and developing software products.
- It is iterative, incremental and focuses on delivering value early and continuously.
- Designed to handle complex and changing requirements.
- Uses short time-boxed iterations called sprints.

2. Scrum Pillars

- Scrum is based on 3 key pillars:

Transparency	Everyone has a shared understanding.
Inspection	Regularly check progress and problems.
Adaptation	Make adjustments when needed.

- These pillars help in continuous improvement.

3. Scrum Values

- Scrum has 5 core values:

Commitment	Dedicated to achieving the goals.
Focus	Focus on the sprint goal.
Openness	Be open about work and challenges.
Respect	Respect team members and their ideas.
Courage	Be brave to do the right thing.

4. Scrum Roles

Product Owner	• Represents the customer. • Maintains the product backlog. • Decides the priority of features.
Scrum Master	• Facilitates the Scrum process. • Helps remove obstacles. • Ensures the team follows Scrum practices.
Development Team	• Cross-functional team. • Self-organizing. • Delivers working software in each sprint.

5. Scrum Events

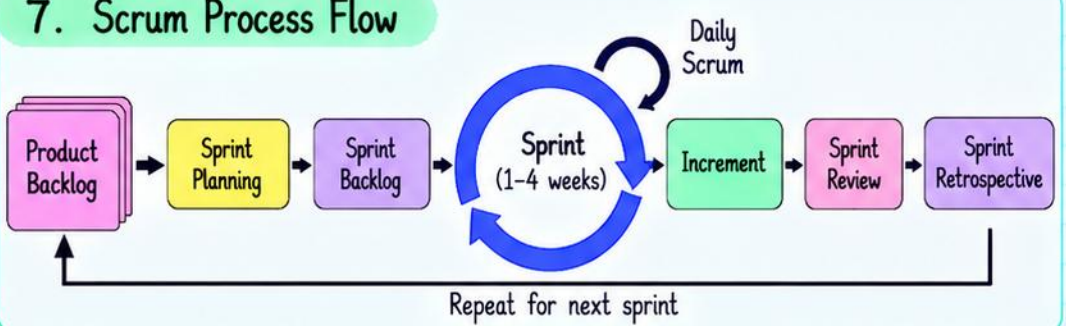
Event	Purpose	Time-box
Sprint	Fixed period to build and deliver an increment.	1–4 weeks
Sprint Planning	Plan the work for the sprint.	Max 8 hours (for 1 month)
Daily Scrum	Short daily meeting to sync and plan.	15 minutes
Sprint Review	Show the completed work to stakeholders.	Max 4 hours (for 1 month)
Sprint Retrospective	Reflect and find improvements.	Max 3 hours (for 1 month)

6. Scrum Artifacts

- Scrum uses 3 main artifacts:

Product Backlog	Ordered list of all requirements/features.
Sprint Backlog	Selected items from product backlog for the current sprint.
Increment	A potentially releasable working product at the end of the sprint.

7. Scrum Process Flow



8. Key Takeaways

- Scrum helps deliver value early and continuously.
- It uses short, time-boxed iterations (sprints).
- Has specific roles, events and artifacts.
- Based on transparency, inspection and adaptation.
- Promotes teamwork, flexibility and continuous improvement.

Scrum Roles in Detail

(Short Notes)

1. Product Owner (PO)

- Represents the customer and stakeholders.
- Responsible for maximizing the value of the product.
- Manages and maintains the product backlog.
- Decides the priority of features.
- Clearly communicates requirements to the development team.
- Accepts or rejects the work done in a sprint.

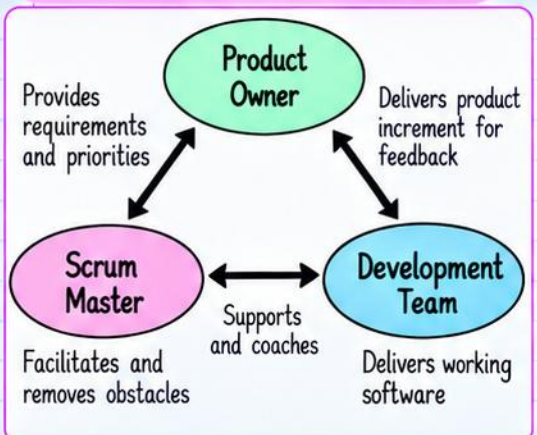
2. Scrum Master (SM)

- Facilitates the Scrum process.
- Helps the team follow Scrum practices and principles.
- Removes obstacles (impediments).
- Ensures effective communication among team members.
- Coaches the team and helps them improve continuously.
- Does not act as a traditional project manager.

3. Development Team

- Cross-functional team with all the skills needed to deliver working software.
- Self-organizing (decides how to do the work).
- Usually 3 – 9 members.
- Works together to complete sprint backlog items.
- Responsible for delivering a potentially releasable product increment at the end of each sprint.

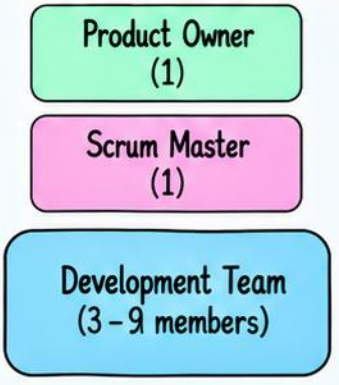
4. Role Collaboration



5. Key Differences

Aspect	Product Owner	Scrum Master	Development Team
Main Focus	Product value	Process and team support	Developing software
Represents	Customer/ Stakeholders	Scrum framework	Themselves
Decision Making	Prioritizes features	Helps team, no product decisions	Decides how to do the work
Key Responsibility	Product backlog	Remove impediments	Deliver product increment
Role Type	Business or customer	Facilitator/ Coach	Technical team

6. Typical Team Structure



7. Example in a Real Project

- **Product Owner**: A university represents students and decides which features to build (e.g. course registration).
- **Scrum Master**: Helps the team follow Scrum, organizes meetings, and removes issues (e.g. tool problems).
- **Development Team**: Includes developers, a UI/UX designer and a tester who work together to build the system.

8. Key Takeaways

- Each role has a clear responsibility.
- All three roles work together to deliver value.
- The Product Owner focuses on what to build.
- The Scrum Master focuses on how to work better.
- The Development Team focuses on building the product.
- A successful Scrum team needs collaboration, trust and communication.

Sprint Planning and Estimation

(Short Notes)

1. What is Sprint Planning ?

- Sprint Planning is a meeting held at the beginning of each sprint.
- The team decides what work will be done in the sprint.
- Selected items from the product backlog are moved to the sprint backlog.
- The team breaks down the work into smaller tasks.
- Everyone discusses and commits to the plan.

2. Sprint Planning Inputs

Product Backlog →	Prioritized list of all features and requirements.
Previous Sprint Performance →	What the team achieved and learned from the last sprint.
Team Capacity →	Availability of team members and their skills.

3. Sprint Planning Outputs

Sprint Goal	A clear short statement of what the team wants to achieve in the sprint.
Sprint Backlog	Selected product backlog items + detailed tasks for the sprint.

The whole team should have a shared understanding of the work to be done.

4. Sprint Planning Process

1. Review Product Backlog	Product Owner presents top priority items.
↓	
2. Discuss and Select Items	Team discusses and selects items.
↓	
3. Set Sprint Goal	Decide what to achieve in this sprint.
↓	
4. Break Down into Tasks	Divide items into smaller tasks.
↓	
5. Estimate the Tasks	Estimate effort for each task (e.g. story points).
↓	
6. Finalize Sprint Backlog	Confirm and commit to the plan.

5. Estimation Techniques

- **Story Points**
  - Relative measure of effort.
  - Based on complexity, not time.
  - Common scale: 1, 2, 3, 5, 8, 13 ...
- **Planning Poker**
  - Team members estimate individually using cards.
  - Discuss large differences and reach a consensus.
- **Other Techniques**
  - T-Shirt sizes (S, M, L, XL)
  - Ideal hours (less common in Scrum)

6. Example – Estimating a User Story

User Story: As a student, I want to register for a course so that I can enroll in the university.

Task	Estimated Story Points
Design registration form	3
Implement form UI	5
Validate input data	3
Save data to database	5
Test registration	3
Total	19

7. Tips for Good Sprint Planning

- Keep the sprint goal realistic and achievable.
- Involve the whole team in discussion.
- Select only what can be completed in the sprint.
- Break down large items into smaller tasks.
- Use relative estimation (story points).
- Be open and honest about challenges.

8. Key Takeaways

- Sprint Planning happens at the start of each sprint.
- Main outputs are the sprint goal and sprint backlog.
- Estimation helps the team understand the effort and plan better.
- Story points are commonly used for estimation.
- A good sprint plan leads to a focused and successful sprint.

Estimation Techniques in Agile

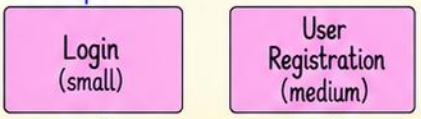
(Short Notes)

1. Why Estimation ?

- Helps in sprint planning.
- Helps to decide what can be completed.
- Provides a realistic view of effort and time.
- Helps in resource allocation.
- Reduces risk of over-commitment.
- Improves transparency and communication.

2. Relative Estimation

- Compares the size/effort of one item with another.
- Uses relative values instead of exact hours.
- Commonly used in Agile (Scrum).
- Example:



3. Planning Poker

- Team members estimate using cards with predefined numbers.
- Discuss differences and reach a consensus.
- Helps in better understanding of requirements.
- Common card values: 0, 1, 2, 3, 5, 8, 13 ...



4. Story Points

- A relative measure of effort.
- Considers complexity, not just time.
- Commonly used in Scrum teams.
- Larger story points = more complex work.
- Example:

User Story	Story Points
Design registration form	3
Implement form UI	5
Validate input data	3
Save data to database	5
Test registration	3

5. T-Shirt Size Estimation

- Uses size categories instead of numbers.
- Useful in early stages when details are limited.
- Common sizes: XS, S, M, L, XL.

Size	Relative Effort
XS	Very small
S	Small
M	Medium
L	Large
XL	Very large

6. Ideal Hours

- Some teams estimate the ideal time (in hours) for a task.
- Does not include interruptions, meetings or unexpected work.
- Less common in Scrum, but still used in some Agile teams.
- Example:

Task	Ideal Hours
Design UI	4
Implement	8
Test	4

7. Comparing Estimation Techniques

Technique	Type	When to Use	Advantages
Relative Estimation	Relative	General use	Simple and effective
Planning Poker	Relative	Team estimation	Encourages discussion
Story Points	Relative	Scrum projects	Considers complexity
T-Shirt Sizes	Relative	Early stages	Quick and easy
Ideal Hours	Absolute	Small/experienced teams	Gives time estimate

8. Key Takeaways

- Estimation helps in sprint planning and decision making.
- Relative estimation is more common in Agile.
- Planning Poker involves the whole team.
- Story points measure complexity, not time.
- T-shirt sizes are useful when details are limited.
- Ideal hours give a time-based estimate.
- Choose the right technique based on your team and project needs.

Task Management in Scrum

(Short Notes)

1. What is a Task ?

- A task is a small, specific piece of work that needs to be done.
- Tasks are created from the selected product backlog items during sprint planning.
- Each task has a clear owner, estimated effort and a purpose.
- Tasks help break down the work so it can be completed during the sprint.

2. Task Characteristics

- Small and doable within the sprint.
- Clearly defined.
- Assigned to a team member.
- Has an estimated effort (e.g. hours).
- Helps achieve the sprint goal.
- Can be updated, moved or reassigned during the sprint.

3. Example Tasks

Product Backlog Item	Example Tasks
User can register for a course	• Design registration form UI • Validate input data • Implement backend logic • Test registration
Admin can add new course	• Create database table • Implement add course form • Validate and save data • Test the feature
User can view available courses	• Design course list page • Fetch data from database • Display course details • Test and fix issues

4. Task Management Tools

- Tasks are usually managed using a task board (physical or digital).
- Common tools:
  - Jira
  - Trello
  - Azure DevOps
  - GitHub Projects
- Teams use columns to track task progress.

5. Example Task Board

To Do	In Progress	Done
Design UI (5h)	Implement backend (6h)	Setup project (2h)
Write tests (3h)	Validate data (3h)	Create database table (4h)
Fix login bug (2h)		

6. Task States

To Do	Task is identified but not started.
In Progress	Task is currently being worked on.
In Review	Task is completed and being reviewed/tested.
Done	Task is finished and accepted.

7. Best Practices for Task Management

- Break large tasks into smaller subtasks.
- Keep task descriptions clear and simple.
- Estimate realistically (use planning poker if needed).
- Update the task board regularly.
- Communicate if there are any blockers.
- Help and support team members when needed.
- Focus on completing the most important tasks first.

8. Key Takeaways

- Tasks are small units of work created during sprint planning.
- They help the team achieve the sprint goal.
- Use a task board to track progress.
- Keep tasks clear, realistic and updated.
- Task management improves teamwork and transparency.
- Effective task management leads to successful sprint delivery.

Scrum Ceremonies in Detail

(Short Notes)

1. Sprint Planning

- Happens at the start of each sprint.
- Team decides what work will be done in the sprint.
- Product Owner presents the top items from the product backlog.
- Team discusses, estimates and selects items.
- Results in a Sprint Backlog.

Time: 1 – 2 hours (for a 2 week sprint)



Team plans the work for the sprint.

2. Daily Scrum (Daily Stand-up)

- Happens every day during the sprint.
- Short meeting (max 15 minutes).
- Each team member answers 3 questions:
  1. What did I do yesterday?
  2. What will I do today?
  3. Are there any blockers?
- Helps the team stay aligned.
- Not a problem-solving meeting (keep it short).

Time: 15 minutes (every day)

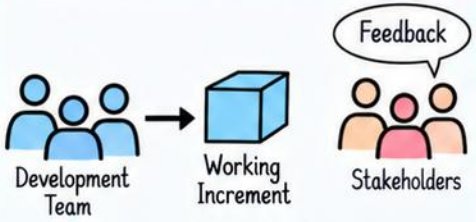


Quick sync to track progress.

3. Sprint Review

- Happens at the end of the sprint.
- Team shows the completed work (increment) to stakeholders.
- Get feedback from stakeholders.
- Discuss what went well and what can be improved.
- May lead to new items in the product backlog.

Time: 1 – 2 hours (for a 2 week sprint)



Show the work and get feedback.

4. Sprint Retrospective

- Happens after the Sprint Review.
- Team reflects on the sprint.
- Discuss what went well, what didn't and how to improve.
- Focuses on the team process, not the product.
- Results in improvement actions for the next sprint.

Common Questions

- What went well?
- What didn't go well?
- What can we do better next time?

Time: 1 hour (for a 2 week sprint)



Continuous improvement.

5. Sprint Ceremony Flow



6. Key Points to Remember

- Scrum has 4 ceremonies:
  1. Sprint Planning
  2. Daily Scrum
  3. Sprint Review
  4. Sprint Retrospective
- Each ceremony has a clear purpose.
- They help with communication, transparency and continuous improvement.
- All team members should actively participate.
- Keep the meetings focused and time-boxed.

7. Key Takeaways

- Scrum ceremonies are time-boxed meetings.
- Each ceremony supports a different part of the sprint.
- They help the team stay aligned, get feedback and improve continuously.
- Following the ceremonies properly leads to a successful and productive Scrum team.

Roles in Scrum and Summary

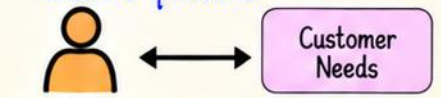
(Short Notes)

1. Scrum Roles Overview

- Scrum has 3 main roles.
- Each role has specific responsibilities.
- All roles work together to achieve the sprint goal.
- There is no traditional "project manager" in Scrum.
- The team is self-organizing and cross-functional.

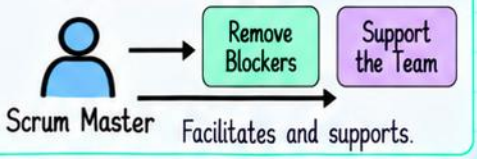
2. Product Owner

- Represents the customer and stakeholders.
- Manages the product backlog.
- Decides the priority of items.
- Ensures the team works on the most valuable features.
- Clarifies requirements and answers questions.



3. Scrum Master

- Facilitates the Scrum process.
- Helps the team follow Scrum practices.
- Removes blockers and obstacles.
- Organizes and supports Scrum ceremonies.
- Ensures continuous improvement.
- Does not manage the team, but serves the team.



4. Development Team

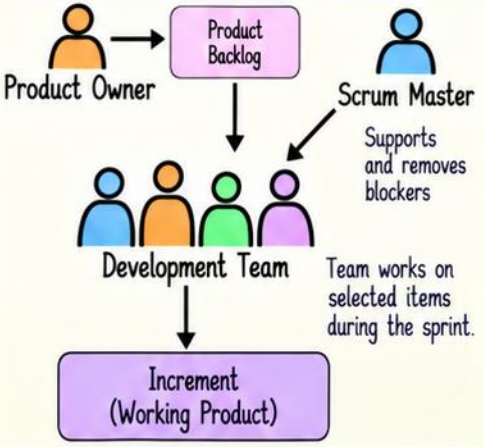
- A small, cross-functional team.
- Does the actual work (design, develop, test, etc.).
- Self-organizing (decides how to do the work).
- Has all the skills needed to complete the sprint goal.
- No sub-teams or hierarchy inside the team.



5. Comparison of Roles

Role	Main Focus	Key Responsibilities
Product Owner	Value (What to build)	• Manage product backlog • Set priority • Represent customer • Clarify requirements
Scrum Master	Process (How to work)	• Facilitate Scrum • Remove blockers • Support the team • Ensure improvement
Development Team	Delivery (Build the product)	• Do the actual work • Self-organizing • Use all required skills • Complete sprint goal

6. Example – How They Work Together



7. Common Misunderstandings

- Scrum Master is not a project manager.
- Product Owner is not the development team leader.
- Development team members are not assigned tasks by the Scrum Master.
- Scrum is not the same as traditional project management.
- Scrum does not remove planning, it makes planning iterative.

8. Key Takeaways

- Scrum has 3 roles: Product Owner, Scrum Master, Development Team.
- Each role has a clear purpose and responsibilities.
- All roles work together through Scrum ceremonies.
- Scrum promotes collaboration, transparency and continuous improvement.
- Understanding the roles helps in successful Scrum implementation.
- Effective teamwork is the key to delivering valuable products.

Benefits and Challenges of Scrum

(Short Notes)

1. Benefits of Scrum (Overview)

- Helps deliver valuable products faster.
- Improves communication and collaboration.
- Provides flexibility to handle changing requirements.
- Increases customer satisfaction.
- Promotes continuous improvement.
- Builds a strong, motivated team.

Scrum = Better teamwork, faster delivery, happier customers.

2. Benefits in Detail

Benefit	Explanation
Faster delivery	Delivers working product in short time (each sprint).
Better communication	Daily Scrum and regular meetings keep everyone informed.
Flexibility	Can adapt to changing requirements easily.
Customer satisfaction	Customer gets working product quickly and can give feedback.
Continuous improvement	Team learns and improves after each sprint.
Team motivation	Self-organizing teams feel more ownership and pride.

3. Challenges of Scrum (Overview)

- Requires active participation from all team members.
- Can be difficult for teams new to Scrum.
- Needs a committed and available Product Owner.
- Frequent meetings may feel time-consuming.
- Does not work well if the organization is not supportive.
- May be challenging for large or distributed teams.

Scrum is powerful, but it also has challenges that must be managed.

4. Challenges in Detail

Challenge	Explanation
Requires active participation	All members must be involved and committed.
Difficult for beginners	Teams may need time to learn and adapt to Scrum practices.
Needs committed Product Owner	If the Product Owner is not available, the sprint can fail.
Meeting overhead	Daily Scrum, Sprint Review, Retrospective can feel time-consuming.
Lack of organizational support	Scrum may not work if the organization prefers traditional methods.
Scaling issues	Can be challenging for very large or distributed teams.

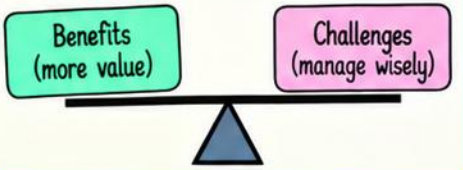
5. Tips to Overcome Challenges

- Provide proper training and understanding of Scrum.
- Ensure the Product Owner is available and committed.
- Keep meetings short and focused.
- Start with a small team and scale gradually.
- Get support from the organization and management.
- Use tools (e.g, Jira, Trello) to manage work and communication.
- Maintain transparency and open communication.

With the right mindset and support, challenges can be turned into opportunities for growth.

6. Summary

- Scrum is a simple but powerful framework for managing software projects.
- It has many benefits such as faster delivery, better collaboration and customer satisfaction.
- However, it also has challenges like meeting overhead and need for active participation.
- Teams should understand both the benefits and challenges to use Scrum effectively.



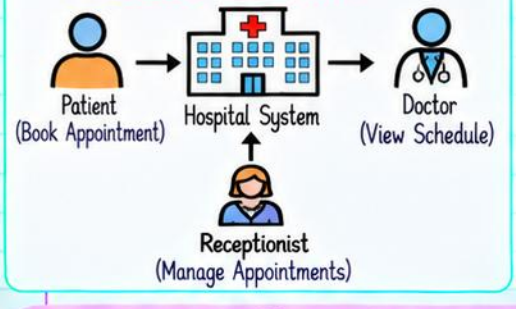
7. Key Takeaways

- Scrum helps teams deliver working products in short sprints.
- It improves communication, flexibility and customer satisfaction.
- There are challenges, but they can be managed with the right approach.
- Success in Scrum depends on committed roles, active teamwork and organizational support.
- Always focus on continuous improvement.

Case Study: Hospital Appointment Management System (Short Notes)

1. Project Overview

- A web-based Hospital Appointment Management System.
- Patients can book, view and cancel appointments.
- Receptionists manage appointments and patient records.
- Doctors can view their schedules.
- Aim: Improve patient experience and reduce manual work.



2. Product Backlog (Prioritized)

No.	Feature / User Story	Priority
1	Book appointment (patient)	High
2	View appointment (patient)	High
3	Cancel appointment (patient)	High
4	Doctor schedule management	Medium
5	Patient registration	Medium
6	Send email/SMS notifications	Medium
7	Admin dashboard (reports)	Low
8	Online payment	Low

Items are prioritized based on customer value, urgency and dependencies.

3. Sprint Planning (Example)

In Sprint Planning, the team selects high-priority items from the Product Backlog for the next sprint.

Sprint Goal (Example)

"Allow patients to book, view and cancel appointments."

Selected Product Backlog Items

No.	Feature
1	Book appointment (patient)
2	View appointment (patient)
3	Cancel appointment (patient)

The team chooses items that can be completed within the sprint (e.g., 2 weeks).

4. Sprint Backlog and Task Breakdown

User Story	Tasks (Sprint Backlog)
Book appointment (patient)	• Design UI for booking form • Develop backend logic • Validate date and time • Test functionality
View appointment (patient)	• Design UI for viewing • Fetch data from database • Display appointment details • Test functionality
Cancel appointment (patient)	• Add cancel button • Update appointment status • Send confirmation message • Test functionality

5. Why High-Priority First?

- Delivers the most valuable features early (MVP).
- Meets critical user needs.
- Gives quick feedback from customers.
- Helps detect problems early.
- If requirements change, high-value features are already delivered.

MVP (Minimum Viable Product)

The simplest working version that delivers the core value to users.

In this example, MVP includes:

- Book appointment
- View appointment
- Cancel appointment

6. Handling Changing Priorities

- In Agile, requirements can change at any time.
- New high-priority items can be added to the product backlog.
- Lower-priority items may be moved to later sprints.
- The team re-evaluates and adjusts in the next sprint planning.

Example:

Later, the hospital requests an urgent feature: "Send SMS notifications".
This new item will be given high priority and added to the next sprint.

7. Key Takeaways

- Product backlog contains all requirements, ordered by priority.
- Sprint planning selects high-priority items for the next sprint.
- Sprint backlog breaks items into smaller tasks.
- High-priority features are chosen first to deliver maximum value.
- MVP helps deliver core functionality early.
- Requirements can change, and Agile handles this through reprioritization.
- Regular feedback from customers ensures the product meets their needs.
- Following this process leads to a successful and useful system.